IBM SKILLSBUILD NATIONAL CYBERSECURITY BOOTCAMP

Conducted by BharatCares (CSRBOX Group) in partnership with IBM SkillsBuild

CAPSTONE PROJECT REPORT • TOPIC 2

Automated Web Scanner: Auditing Sites for Secure Headers & CORS Flaws

A practical, zero-dependency Python tool developed to automatically evaluate web application HTTP security headers and detect dangerous Cross-Origin Resource Sharing (CORS) misconfigurations.

PARTICIPANT / AUTHOR

Tanisha

GitHub: tsh471

Track: Cybersecurity & Web Defense

BOOTCAMP DETAILS

IBM SkillsBuild National Cybersecurity Bootcamp

Submission: October 2026

1. Project Overview & Motivation

During the 2-week IBM SkillsBuild National Cybersecurity Bootcamp, we covered a variety of cybersecurity fundamentals, including threat modeling, ethical hacking principles, and network security. For my final project submission, I selected **Topic #2: Automated Web Scanner: Programmed a Python script to audit sites for secure headers and CORS flaws**.

In modern web development, teams often focus entirely on user interfaces and database logic while overlooking fundamental HTTP transport and access control configurations. In fact, *Security Misconfiguration* ranks prominently under **OWASP Top 10 (A05:2021)**. Failing to configure defensive HTTP headers leaves users open to clickjacking, unauthorized script execution (XSS), and protocol downgrade attacks. Furthermore, when configuring Cross-Origin Resource Sharing (CORS) to connect frontend single-page apps to backend APIs, developers frequently take insecure shortcuts—such as reflecting incoming `Origin` headers indiscriminately or combining wildcards with credentials.

To tackle this issue, I programmed an automated Python security scanner. The tool takes any web address, analyzes its HTTP response headers, actively probes the endpoint with synthetic origin headers to detect CORS flaws, calculates an objective 0–100 security score with letter grades (A+ to F), and outputs clear reports in the terminal, as structured JSON, or as an HTML report.

> **Key Design Philosophy**
> I built the tool with **zero external dependencies** using only Python's built-in standard library (`urllib.request` , `concurrent.futures` , `json` , `argparse` , `ssl`). This ensures anyone can run the scanner immediately on any machine running Python 3 without dealing with package installation issues.

2. Objectives & Design Choices

I set three clear design goals for this project:

- **Comprehensive Defensive Audit:** Check all critical security headers specified by the OWASP Secure Headers Project (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy), plus information disclosure banners like `Server` and `X-Powered-By`.
- **Active Synthetic CORS Probing:** Rather than just passively reading existing CORS headers, the scanner actively tests how the server behaves when probed with untrusted origins (e.g. an attacker domain, `null`, and regex prefix/suffix bypass attempts).
- **Actionable Feedback & Scoring:** The scanner gives developers practical remediation snippets (such as exact Nginx configuration lines or Flask code) so they can fix flagged weaknesses immediately.

3. Technical Methodology

3.1 HTTP Security Headers Audit

The scanner's `HeaderAuditor` evaluates the target's HTTP responses against modern web security standards:

Header Name	Recommended Baseline	Risk if Missing or Misconfigured
Content-Security-Policy	`default-src 'self'; script-src 'self'; object-src 'none';`	High risk of Cross-Site Scripting (XSS) and unauthorized data exfiltration.
Strict-Transport-Security	`max-age=31536000; includeSubDomains; preload`	Allows SSL-stripping and cleartext downgrade attacks over untrusted Wi-Fi.
X-Frame-Options	`DENY` or `SAMEORIGIN`	Vulnerable to Clickjacking (UI redressing) in legacy browsers.
X-Content-Type-Options	`nosniff`	Browsers may execute uploaded non-executable files via MIME sniffing.
Referrer-Policy	`strict-origin-when-cross-origin`	Leaks sensitive tokens or internal URLs to external third parties.
Permissions-Policy	`camera=(), microphone=(), geolocation=()`	Browser hardware features remain open to unauthorized access.
Server & Tech Banners	Suppress `Server` and `X-Powered-By`	Reveals exact server OS and software versions to automated scanners.

3.2 CORS Flaw Detection & Probing

Cross-Origin Resource Sharing (CORS) relaxes the Same-Origin Policy. To audit CORS logic, the scanner sends active test probes:

- **Arbitrary Origin Reflection:** The scanner sends `Origin: https://attacker-security-test.com`. If the server echoes this origin in `Access-Control-Allow-Origin`, it reveals flawed origin validation.
- **Origin Reflection with Credentials (CRITICAL):** If arbitrary reflection is combined with `Access-Control-Allow-Credentials: true`, this is marked CRITICAL. An attacker site can issue cross-origin requests using a victim's session cookies and read private API responses.
- **Null Origin Trust (HIGH):** The scanner tests `Origin: null`. If accepted, attackers can bypass origin checks using sandboxed `<iframe>` elements or `data:` URIs.

- **Regex Suffix & Prefix Bypasses:** Tests whether unanchored regular expressions allow origins like `https://target.com.attacker.com` or `https://attacker-target.com`.

3.3 Scoring & Grade Calculation

The audit starts with a perfect score of 100 points. Penalties are deducted based on risk severity: Missing CSP (-25), Missing HSTS (-20), Critical CORS Credential Reflection (-30), High CORS Flaw / Null Origin (-20), Missing X-Frame-Options (-15), Missing X-Content-Type-Options (-10), and Leaky Banners (-5). Scores translate directly into letter grades: **A+ (95–100)**, **A (85–94)**, **B (75–84)**, **C (60–74)**, **D (45–59)**, and **F (<45)**.

4. Code Architecture & Implementation

The script is organized in a modular structure within `scanner.py`:

- `HeaderAuditor`: Encapsulates header evaluation logic and severity grading.
- `CORSAuditor`: Sends synthetic HTTP requests with test origin headers and examines server responses.
- `WebScanner`: Handles target requests, thread pooling (`concurrent.futures.ThreadPoolExecutor`) for scanning multiple URLs simultaneously, and final score calculations.
- `Reporter`: Formats scan data into colored terminal output, clean JSON, or standalone HTML.
- `--demo Mode`: A built-in offline demonstration mode that simulates both a vulnerable and a hardened target so evaluators can test the tool without needing live network access.

5. Testing & Experimental Results

I validated the tool using both automated unit testing and sample scans:

5.1 Unit Tests (tests/test_scanner.py)

Running the unit test suite passes all 5 tests instantly:

```
python3 -m unittest discover -s tests -p "test_*.py" -v
test_information_leakage_detection ... ok
test_missing_all_security_headers ... ok
test_strong_security_headers ... ok
test_weak_csp_warning ... ok
test_grade_calculation ... ok
Ran 5 tests in 0.000s (OK)
```

5.2 Sample Audit Output on a Vulnerable Endpoint

```
----------------------------------------------------------------------
[1/1] Target: https://vulnerable-portal.local/api
Scan Time: 2026-10-24 15:46:57 | HTTP Status: 200 | Latency: 142.5 ms
Security Score: 35/100 (Grade: F)
----------------------------------------------------------------------
--- [1] HTTP SECURE HEADERS AUDIT ---
  [FAIL] Content-Security-Policy (Severity: HIGH)
         Missing CSP header. Vulnerable to XSS and Clickjacking.
  [FAIL] Strict-Transport-Security (Severity: HIGH)
         Missing HSTS header on HTTPS connection.
  [FAIL] X-Frame-Options (Severity: MEDIUM)
         Missing X-Frame-Options. Page can be framed for Clickjacking.
  [PASS] X-Content-Type-Options (Severity: PASS) - nosniff enabled.
--- [2] CORS MISCONFIGURATION PROBING ---
  [CRITICAL] Arbitrary Origin Reflection with Credentials
```

```
        Test Origin: https://attacker-security-test.com
        Finding: Server reflects arbitrary Origin AND sets Access-Control-Allow-Credentia
        ACAO: https://attacker-security-test.com | ACAC: true
[HIGH] Null Origin Trust
        Test Origin: null | Finding: Server explicitly trusts 'null' origin.
```

6. Challenges Faced & Solutions

During the project, I addressed several practical challenges:

- **Handling SSL Certificate Verification:** Testing local development servers or self-signed staging environments often throws SSL verification errors. I added a `-k / --insecure` command-line flag to let testers bypass certificate checks when auditing lab environments.
- **User-Agent Filtering:** Several web applications block the default Python-urllib User-Agent. I solved this by configuring a realistic browser User-Agent in the HTTP request headers.
- **Distinguishing Safe Wildcards from Insecure CORS:** A wildcard `Access-Control-Allow-Origin: *` is completely fine for public read-only content, but dangerous when credentials are enabled. I ensured the scanner only flags high-severity warnings when wildcards or reflected origins are paired with `Access-Control-Allow-Credentials: true`.

7. Remediation & Hardening Guide

Fixing issues flagged by the scanner is straightforward for developers:

Nginx Server Configuration

```
# /etc/nginx/conf.d/security_headers.conf
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'no
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
server_tokens off;
```

Secure Backend CORS Whitelist (Python / Flask)

```
ALLOWED_ORIGINS = {"https://trusted-portal.com", "https://admin.trusted-portal.com"}

@app.after_request
def set_secure_cors(response):
    origin = request.headers.get("Origin")
    if origin in ALLOWED_ORIGINS:
        response.headers["Access-Control-Allow-Origin"] = origin
        response.headers["Access-Control-Allow-Credentials"] = "true"
        response.headers["Vary"] = "Origin"
    return response
```

8. Key Learnings & Conclusion

This project provided great hands-on learning during the IBM SkillsBuild Cybersecurity Bootcamp. I learned how easily security misconfigurations can occur in real-world web environments, how the browser's Same-Origin Policy interacts with CORS headers, and why automated auditing is necessary to catch vulnerabilities before code is pushed to production.

The resulting Python scanner is fast, portable, and reliable. It serves as an effective auditing utility for developers and security analysts to ensure web applications maintain strong baseline security.

References:

1. OWASP Top 10 (A05:2021 - Security Misconfiguration)
2. OWASP Secure Headers Project Guidelines
3. W3C Cross-Origin Resource Sharing (CORS) Specification & MDN Web Docs
4. IBM SkillsBuild National Cybersecurity Bootcamp Masterclasses (October 2026)